



Introduction to Software Testing

Programming and Development of Embedded Systems

Software Testing

❖ What is Software Testing?

- The process to evaluate functionalities of a software in order to
 - Find whether the developed software fulfills the specified requirements or not
 - Identify defects to ensure that the software is defect free

❖ Why is Software Testing Important?

- Cost effectiveness
 - Identifying and fixing a bug early costs much less to fix it later
- Customer Satisfaction
 - Customer satisfaction is the ultimate goal of any business
 - Software testing improves the user experience of an application

Software Testing

❖ Why is Software Testing Important?

➤ Safety and Security

- Testing helps to remove vulnerabilities in products
- China Airlines Airbus A300 crashed due to a software bug, killing 264 lives (1994)

➤ Product Quality

- Testing helps to deliver high quality products to clients according to requirements

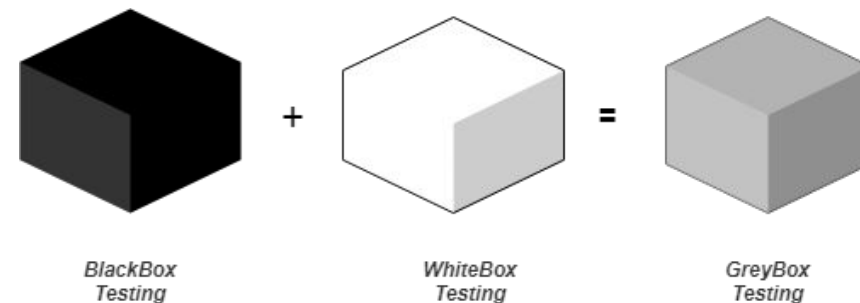
❖ Testing Methods:

➤ White Box Testing

➤ Black Box Testing

➤ Grey Box Testing

- It is a combination of white-box testing and black-box testing
- It is primarily used in Integration Testing



Black Box Testing

- ❖ It is also known as **Behavioral** testing
- ❖ The internal structure/design/implementation of the component/system is unknown
- ❖ The higher the level, bigger and more complex the box, the more black-box testing is used
- ❖ It can be used in the functional and non-functional testings

- ❖ Some Techniques:

- Boundary Value Analysis

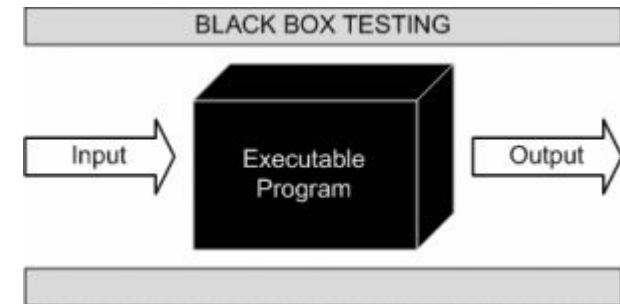
- Involves the determination of boundaries for input values
 - Using values that are at the boundaries and just inside/outside of the boundaries as test data

- Equivalence Partitioning

- Dividing input values into valid and invalid partitions and using them as the test data

- Cause-Effect Graphing:

- Involves identifying the cases that produce a **Cause-Effect Graph**, and generating test cases



Black Box Testing

❖ Black Box Testing can be used to find errors in

- Incorrect or missing functions, Interface errors and errors in data structures or external database
- Behavior or performance errors, and initialization and termination errors

❖ Black Box Testing Advantages

- Tests are done from a user's point of view
- Tests can be done by a body independent of developers
- Tester does not need to know programming languages or how the software has been implemented
- Test cases can be designed as soon as the specifications are complete



[What is Black Box Testing?](#)

❖ Black Box Testing Disadvantages

- Only a small number of possible inputs can be tested and many program paths will be left untested
- Without clear specifications, which is the case in many projects, test cases will be difficult to design
- Tests can be redundant if the software designer/developer has already run a test case

White Box Testing

- ❖ The internal structure/design/implementation of the component/system is known
- ❖ Testing is based on an analysis of the internal structure of the component or system
- ❖ White Box Testing Advantages
 - Testing is more complete, with the possibility of covering most paths
 - Code optimization by finding hidden errors
 - Testing can be started at an earlier stage
 - Test cases can be easily automated
- ❖ White Box Testing Disadvantages
 - It can be quite complex and expensive (highly skilled resources are required)
 - Test script maintenance can be a burden if the implementation changes too frequently
 - Since it is closely tied to the application under test, tools for every kind of implementation/platform may not be available



White Box Testing

❖ White Box Testing Techniques

- Statement Coverage: Every possible statement in the code should be tested at least once during the testing
- Branch Coverage: It checks every possible path (if-else and other conditional loops) of a software



[What is White Box Testing? Tutorial with Examples](#)

❖ What do you verify in White Box Testing?

- Internal security holes
- The flow of specific inputs through the code
- Expected output
- The functionality of conditional loops
- Testing of each statement, object, and function on an individual basis
- Broken or poorly structured paths in the coding processes

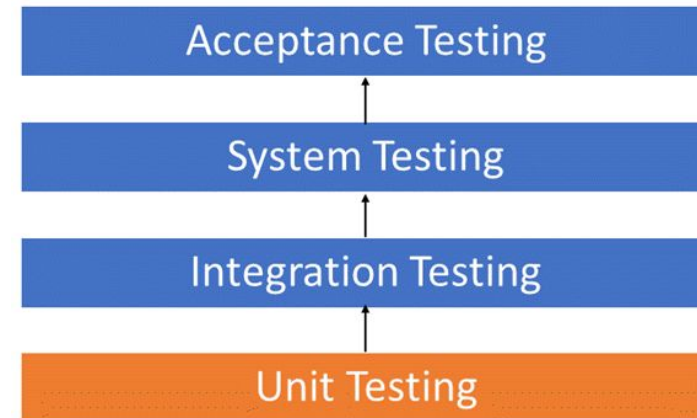
Testing Levels

❖ Software Testing Levels

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

❖ Types of Software Testing

- Functional Testing
- Non-Functional Testing
- Maintenance Testing
- And etc.



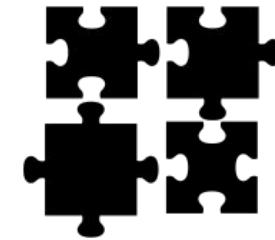
[Types of Software Testing: 100 Examples of Different Testing Types](#)

Unit Testing

- ❖ The first level of testing; Individual units/components of a software are tested
- ❖ The purpose is to validate that each unit of the software performs as designed
- ❖ A unit is the smallest testable part of a software (e.g. functions, components and etc.)
- ❖ It is performed using the **White Box Testing** method
- ❖ It is normally performed by software developers themselves
- ❖ Unit Testing Benefits
 - It increases confidence in changing/maintaining code (they are run every time any code is changed)
 - Codes are more reusable. In order to make unit testing possible, codes need to be modular.
 - Development is faster (short term and long term)
 - Writing tests takes lesser time than the time it takes to run the tests
 - It takes lesser time to find and fix bugs than integration, system or acceptance testing
 - Debugging is easy. When a test fails, only the latest changes need to be debugged



[What is Unit Testing? - Software Testing Tutorial](#)

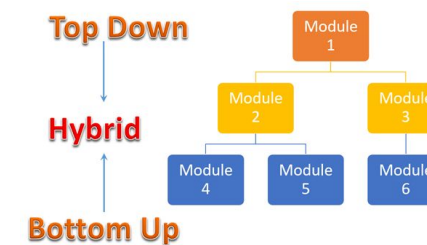
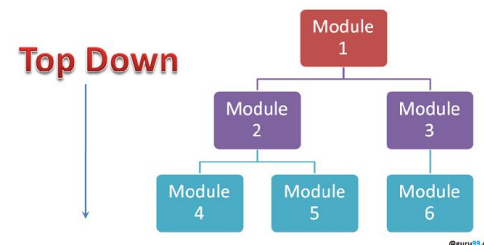
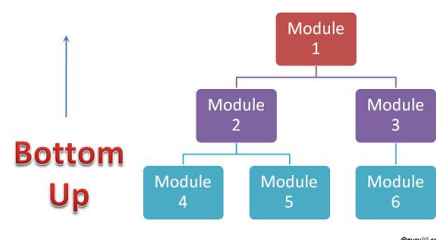
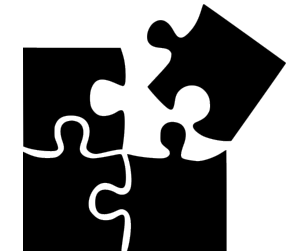


Integration Testing

- ❖ It is a level of software testing where individual units are combined and tested as a group
- ❖ The purpose of this level of testing is to expose faults in the interaction between integrated units.
- ❖ Any of Black Box, White Box and Gray Box Testing methods can be used
- ❖ It is done by developers themselves or independent testers
- ❖ Integration Testing Approaches
 - **Big Bang:** Here all components are integrated together at once and then tested
 - **Incremental:** Testing is done by joining two or more modules that are logically related
 - Bottom-up Integration, Top-down Integration and Hybrid/ Sandwich Integration



[What is Integration Testing?](#)



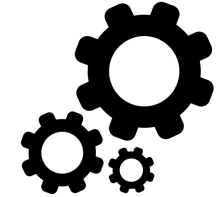
System and Acceptance Testing

❖ System Testing



[What is System Testing?](#)

- The process of testing a fully integrated system to verify that it meets the system level requirements
- Usually, Black Box Testing method is used and it is done by Independent testers
- E.g. Usability Testing, Load Testing, Hardware/Software Testing and etc.



❖ Acceptance Testing

- Formal testing with respect to user requirements and business processes to determine whether or not a system satisfies the customer acceptance criteria
- Usually Black Box Testing is used. It is done by people who are not directly involved in the development of the product and can be done:
 - Internally: by members of the organization
 - External: by the customer or customer's of the customer (end users)

Test-Driven Development (TDD)

- ❖ Test-Driven Development is a technique to develop softwares incrementally
 - Small steps help form better components
- ❖ A failing test is followed immediately by code satisfying the test
 - Healthy code growth, components are less prone of bugs.
- ❖ The focus is on the requirements instead of coding
 - Tests shall be based on the requirements
 - Tests shall enforce a specific behaviour
 - Tests shall encourage refactoring and make components easier to understand
 - Refactoring is the process of restructuring code without changing its external behavior.
- ❖ Test automation is key to TDD
 - Testing is automatically performed by machine over and over. No need for manual testing.

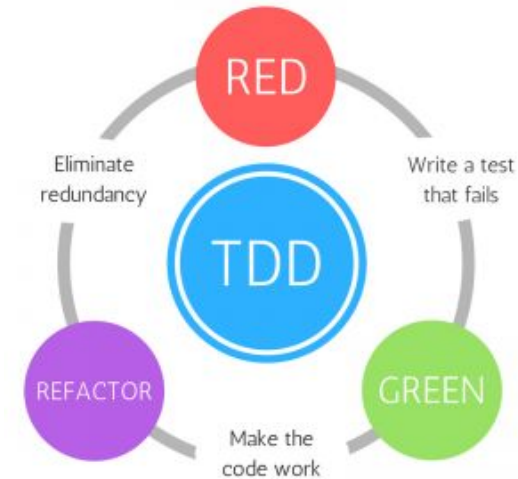
Test-Driven Development (TDD)

❖ Three rules of TDD

- Write production code only to pass a failing unit test.
- Write no more of a unit test than sufficient to fail.
 - Compilation failures are failures
- Write no more production code than necessary to pass the one failing unit test.

❖ The steps of the TDD cycle

- Add a small test and run all the tests to ensure that the new test fails
- Make small changes needed to pass the failing test
- Run all the tests and ensure that the new test is passed
- Refactor to remove duplication and improve expressiveness
- After refactoring run all the tests to ensure that the tests are passed



Test-Driven Development - TDD

❖ TDD Benefits

- Fewer bugs: Small and large logic errors are found quickly during development
- Less debug time: Having fewer bugs means less debug time.
- Fewer side effect defects: When new code violates a constraint or assumption, the tests scream
- Documentation
 - Well-structured tests become a form of executable documentation
- Improved design
 - A good design is a testable design.
 - You get an early warning of design problems if tests cannot be written for the code change
- Progress monitor
 - The tests keep track of exactly what is working and how much work is done

Test-Driven Development - TDD

❖ General testing guidelines

- Tests should
 - Be easy to follow
 - Help to maintain healthiness of the code
 - Tell us when something is changed
 - Be derived from requirements
 - Enforce a specific behaviour
 - Encourage refactoring
- Different test cases require different test types
 - Evaluate the best test type for your tests
 - Tests can derive into multiple test types

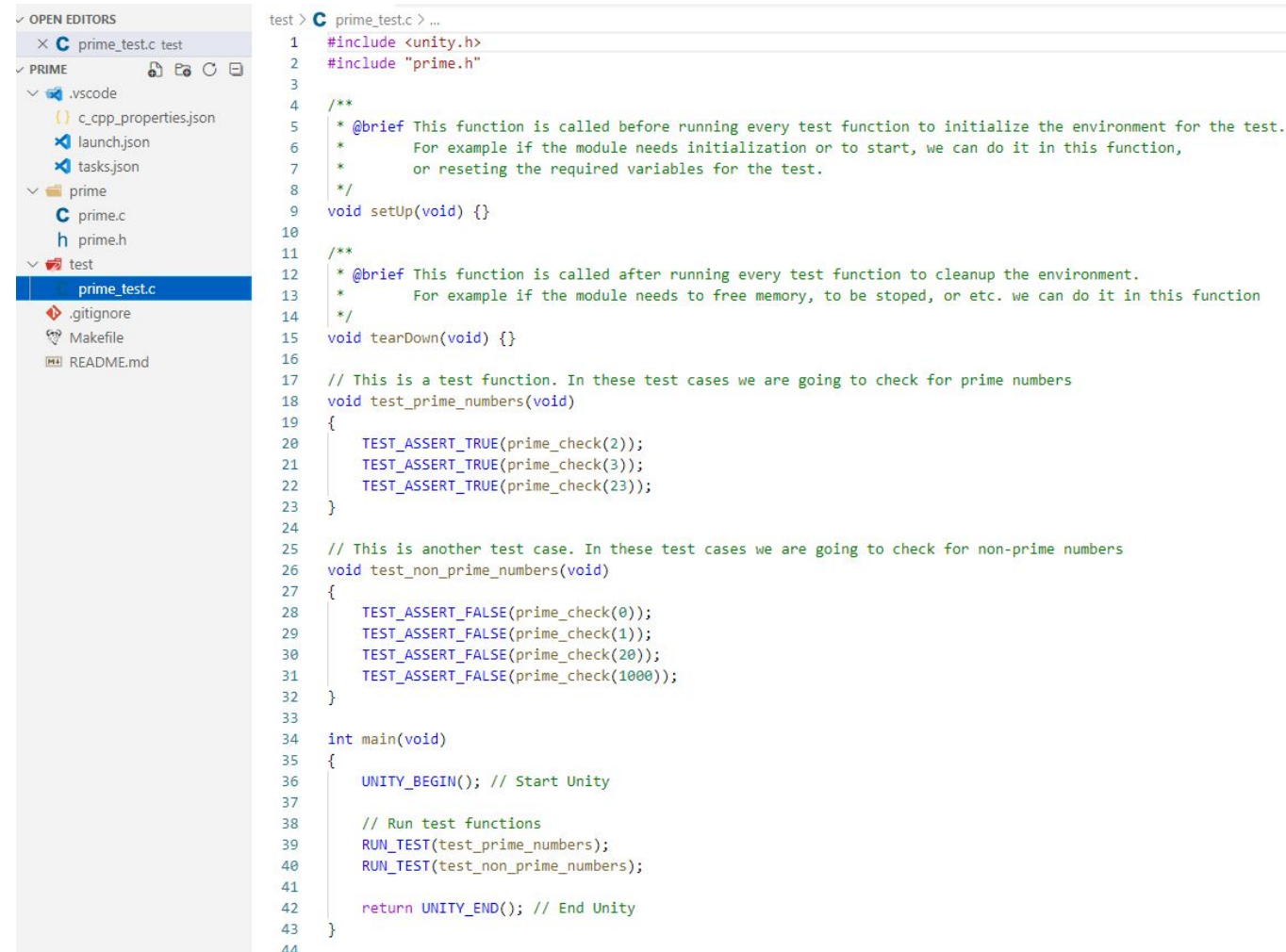
GDB

- ❖ Debugging is the process of finding and solving problems within a program using a debugger.
- ❖ **GDB** is the [GNU Project debugger](#) which can be used to debug programs written in C/C++ languages.
- ❖ To compile your program for debugging you need to use **-g**. E.g. **`gcc -g main.c -o main`**
- ❖ You can use **gdb** to debug your program in [command line](#) or in Visual studio code.
- ❖ To use gdb & gcc to build, debug and run in visual studio code
 - Click on the run icon and then click on the **Create a launch.json file**.
 - Select C++(GDB/LLDB) in the opened list. And then click on **Add Configuration** and Select **C/C++ (gdb) Launch**
 - Then change: "program": "**path/to/your/program**"
 - Then set the path of the debugger(gdb): "miDebuggerPath": "**gdb**"
 - To make a task to compile & run your program click on **Terminal > Configure Tasks..** in the main menu.
 - Then click on **Create tasks.json file from template** and then **Others Examples to run an arbitrary external command**.
 - The **tasks.json** file is created. Change the label from **echo** to **Build**
 - In the **command** write shell commands to build your program.
 - Then open **launch.json** and add "preLaunchTask": "**Build**" to the configurations.

Unity

- ❖ [Unity](#) is a portable unit testing framework written in standard C and supports testing of embedded systems. [Unity Assertions Reference](#)
- ❖ Unity uses assertions to test the actual values and the expectations
- ❖ Assertions are statements of what we expect to be true about the code under test.
 - E.g. `TEST_ASSERT_EQUAL_UINT8(expected, actual);`
 - Checks if the expected value and the actual value are equal or not
 - If they are not equal, it means the test is failed
- ❖ Unity is used to test modules. Means that the code under test shall be module(s).
 - A simple module is a source and a header files.
 - To test a module you also need to create a test source file and include `unity.h`
 - The test cases are implemented in this file.

Unity - Example



The image shows a Visual Studio Code editor window. On the left, the Explorer sidebar displays a project structure for a C project named 'PRIME'. The project files include a '.vscode' folder with 'c_cpp_properties.json', 'launch.json', and 'tasks.json'; a 'prime' folder with 'prime.c' and 'prime.h'; and a 'test' folder with 'prime_test.c'. The 'prime_test.c' file is currently open in the editor. The code in 'prime_test.c' is a C program using the Unity testing framework. It includes 'unity.h' and 'prime.h'. It defines two test functions: 'test_prime_numbers' which asserts that 2, 3, and 23 are prime numbers, and 'test_non_prime_numbers' which asserts that 0, 1, 20, and 1000 are not prime numbers. The 'main' function initializes Unity, runs the test functions, and returns 'UNITY_END()'.

```
test > C prime_test.c > ...
1  #include <unity.h>
2  #include "prime.h"
3
4  /**
5   * @brief This function is called before running every test function to initialize the environment for the test.
6   *        For example if the module needs initialization or to start, we can do it in this function,
7   *        or resetting the required variables for the test.
8   */
9  void setUp(void) {}
10
11 /**
12  * @brief This function is called after running every test function to cleanup the environment.
13  *        For example if the module needs to free memory, to be stoped, or etc. we can do it in this function
14  */
15 void tearDown(void) {}
16
17 // This is a test function. In these test cases we are going to check for prime numbers
18 void test_prime_numbers(void)
19 {
20     TEST_ASSERT_TRUE(prime_check(2));
21     TEST_ASSERT_TRUE(prime_check(3));
22     TEST_ASSERT_TRUE(prime_check(23));
23 }
24
25 // This is another test case. In these test cases we are going to check for non-prime numbers
26 void test_non_prime_numbers(void)
27 {
28     TEST_ASSERT_FALSE(prime_check(0));
29     TEST_ASSERT_FALSE(prime_check(1));
30     TEST_ASSERT_FALSE(prime_check(20));
31     TEST_ASSERT_FALSE(prime_check(1000));
32 }
33
34 int main(void)
35 {
36     UNITY_BEGIN(); // Start Unity
37
38     // Run test functions
39     RUN_TEST(test_prime_numbers);
40     RUN_TEST(test_non_prime_numbers);
41
42     return UNITY_END(); // End Unity
43 }
44
```

Introduction to Software Testing

❖ Some useful links

- [Software Testing Tutorial: Free Course](#)
- [Software Testing Fundamentals](#)
- [The Three Laws of TDD](#)
- [What Is Software Testing?](#)
- [The Three Laws of TDD \(Featuring Kotlin\)](#)
- [How TDD is related to the quality of code](#)
- [What is Test Driven Development?](#)
- [What is Behavior Driven Development?](#)
- [TDD vs BDD. Giants of Test Go Head-to-Head \(1\)](#)
- [TDD vs BDD. The Giants of Test Go Head-to-Head \(2\)](#)
- [TDD vs BDD - Your free CHEAT SHEET](#)
- [TDD and BDD - What Are The Key Differences?](#)